Tugas 7
Implementasi AI untuk Keuangan – Laporan Pengeluaran Mingguan

Arief Rachman Apriansyah - 0110224166
Grup 3
TEKNIK INFORMATIKA 2024

Sekolah Tinggi Teknologi Terpadu Nurul Fikri
Depok
2026

1. Pendahuluan

1.1 Latar Belakang

Mencatat pengeluaran saja belum cukup; nilainya baru terasa ketika catatan itu ditinjau secara rutin. Peninjauan manual setiap minggu memakan waktu dan mudah terlewat. Karena itu dibangun workflow n8n yang setiap hari Minggu pagi mengambil log pengeluaran dari Google Sheets, menyaring transaksi 7 hari terakhir, meminta AI menganalisis kebiasaan belanja beserta satu saran penghematan, lalu mengirimkan hasilnya ke Telegram pribadi.

1.2 Tujuan

- Mengintegrasikan Google Sheets sebagai sumber data log transaksi dalam alur otomasi yang dipicu jadwal mingguan.
- Menyusun skrip JavaScript untuk menyaring data berdasarkan rentang 7 hari terakhir dan memformat teks laporan.
- Menerapkan logika percabangan berdasarkan ketersediaan data, termasuk pesan "Nihil" bila tidak ada pengeluaran.
- Memanfaatkan AI (Google Gemini) untuk memberi wawasan analisis dan satu saran penghematan konkret.

2. Gambaran Umum Workflow

Workflow bernama "Laporan Pengeluaran Mingguan" terdiri dari delapan node. Alurnya: Schedule Trigger memulai proses, Google Sheets mengambil seluruh data, node Code menyaring dan memformat data, node If memeriksa ketersediaan data, lalu alur bercabang ke pesan Nihil (False) atau ke analisis AI dan pengiriman laporan (True). Susunan lengkapnya terlihat pada Gambar 1.

No	Node	Fungsi	Konfigurasi Utama
1	Setiap Hari Minggu (Schedule Trigger)	Memicu workflow otomatis	Interval mingguan, hari Minggu, pukul 07.00
2	Ambil Log Pengeluaran (Google Sheets)	Mengambil seluruh baris transaksi	Operation Get Row(s), Sheet1, Always Output Data aktif
3	Filter 7 Hari & Format Teks (Code)	Menyaring 7 hari terakhir dan menyusun teks	JavaScript, mode Run Once for All Items
4	If	Memeriksa ketersediaan data	hasExpenses is true
5	Pesan False (Telegram)	Mengirim notifikasi "Nihil"	Send Message
6	Analisis (Basic LLM Chain)	Meminta AI merangkum dan memberi saran	System message asisten keuangan, prompt dinamis
7	Google Gemini Chat Model	Model bahasa untuk analisis	Gemini Flash Lite, temperature 0.4
8	Pesan True (Telegram)	Mengirim hasil analisis ke pengguna	Send Message ke chat pribadi

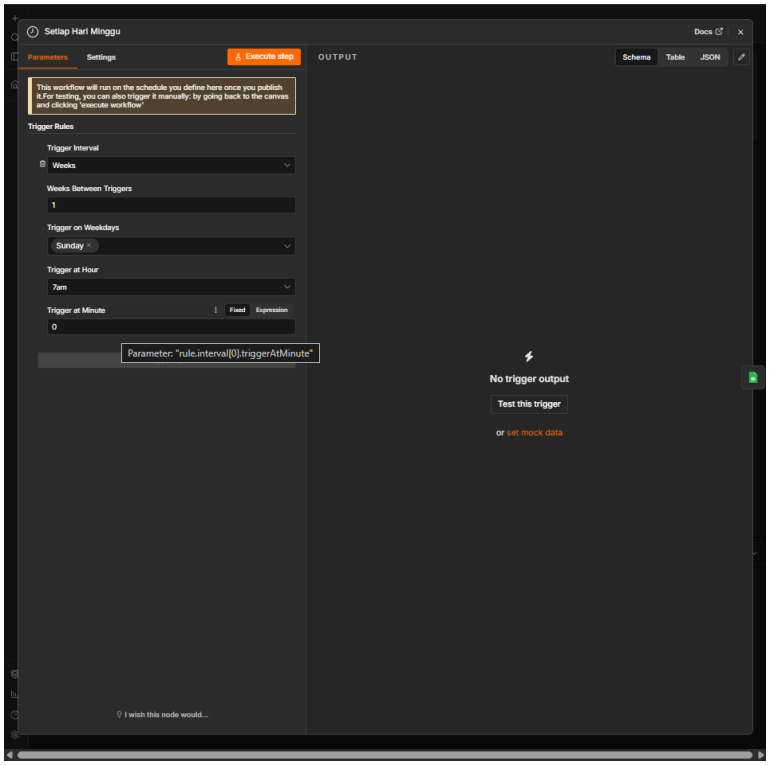


Gambar 1. Tampilan keseluruhan workflow Laporan Pengeluaran Mingguan

3. Implementasi Setiap Tugas

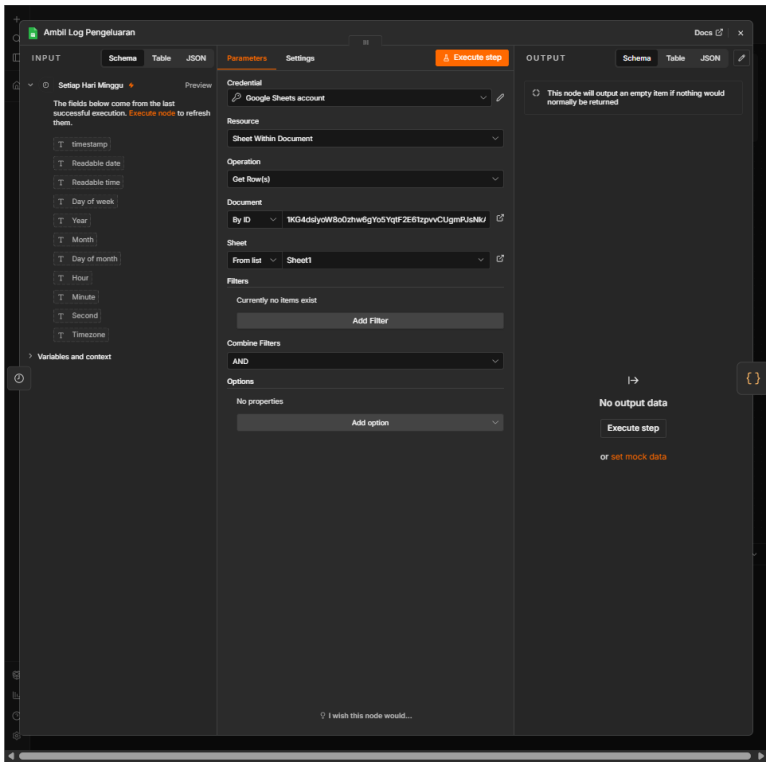
3.1 Tugas 1: Schedule Trigger dan Pengambilan Data dari Google Sheets

Node Setiap Hari Minggu memakai Schedule Trigger dengan interval Weeks, setiap 1 minggu, pada hari Sunday, pukul 7am dan menit 0 (Gambar 2). Dengan begitu laporan dibuat otomatis setiap Minggu pagi pukul 07.00.



Gambar 2. Konfigurasi node Schedule Trigger (setiap Minggu pukul 07.00)

Node Ambil Log Pengeluaran menggunakan credential Google Sheets dengan operasi Get Row(s) pada dokumen "Data Dummy Catatan Keuangan" (dipilih By ID) dan sheet Sheet1, tanpa filter sehingga seluruh baris terambil (Gambar 3). Pada pengaturan node, opsi Always Output Data diaktifkan agar node tetap mengeluarkan satu item kosong jika sheet tidak berisi data, sehingga jalur "Nihil" tetap dapat berjalan.



Gambar 3. Konfigurasi node Ambil Log Pengeluaran (Get Row(s) dari Sheet1)

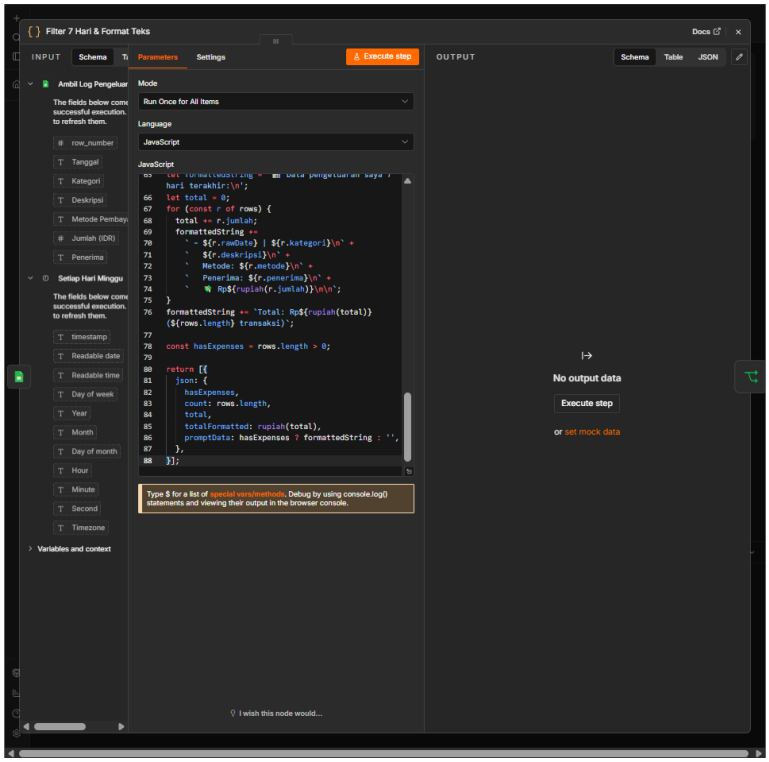
3.2 Tugas 2: Skrip JavaScript Filter 7 Hari Terakhir

Node Filter 7 Hari & Format Teks (mode Run Once for All Items) menjalankan dua tugas: menyaring transaksi dan memformatnya menjadi teks untuk AI. Kode lengkap ada di Lampiran A, sedangkan bagian akhirnya terlihat pada Gambar 4. Poin pentingnya:

- Rentang waktu: batas atas adalah hari ini pukul 23:59:59 dan batas bawah adalah 7 hari sebelumnya pukul 00:00:00, sehingga seluruh transaksi hari ini ikut terhitung.
- Format tanggal DD/MM/YYYY diurai manual dengan regex dan dibentuk menjadi objek Date (new Date(year, month - 1, day)), supaya JavaScript tidak salah menafsirkan hari dan bulan.
- Nama kolom dibaca dengan fungsi pick() yang mencocokkan awalan nama header tanpa membedakan huruf besar/kecil, sehingga "Jumlah (IDR)" dan "Metode Pembayaran" terbaca dengan benar.
- Jumlah diubah menjadi angka (teks seperti "Rp 25.000" tetap terbaca), lalu total dihitung dan ditampilkan dengan pemisah ribuan.
- Output berupa satu item berisi hasExpenses, count, total, totalFormatted, dan promptData (teks terstruktur untuk AI).

```
const today = new Date();
today.setHours(23, 59, 59, 999);
const sevenDaysAgo = new Date(today);
sevenDaysAgo.setDate(today.getDate() - 7);
sevenDaysAgo.setHours(0, 0, 0, 0);

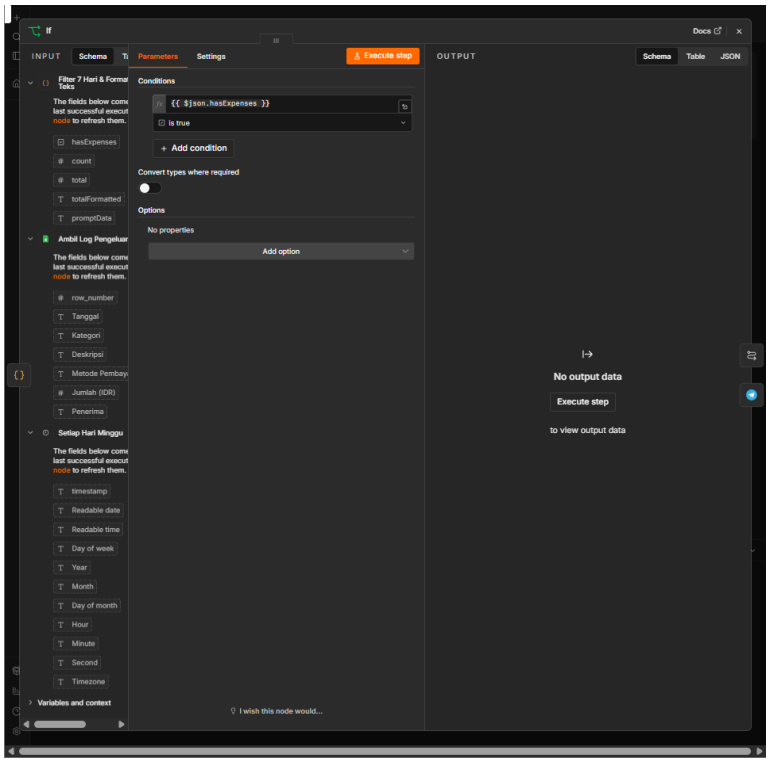
if (expenseDate >= sevenDaysAgo && expenseDate <= today) { ... }
```



Gambar 4. Node Filter 7 Hari & Format Teks: bagian akhir kode dan struktur output

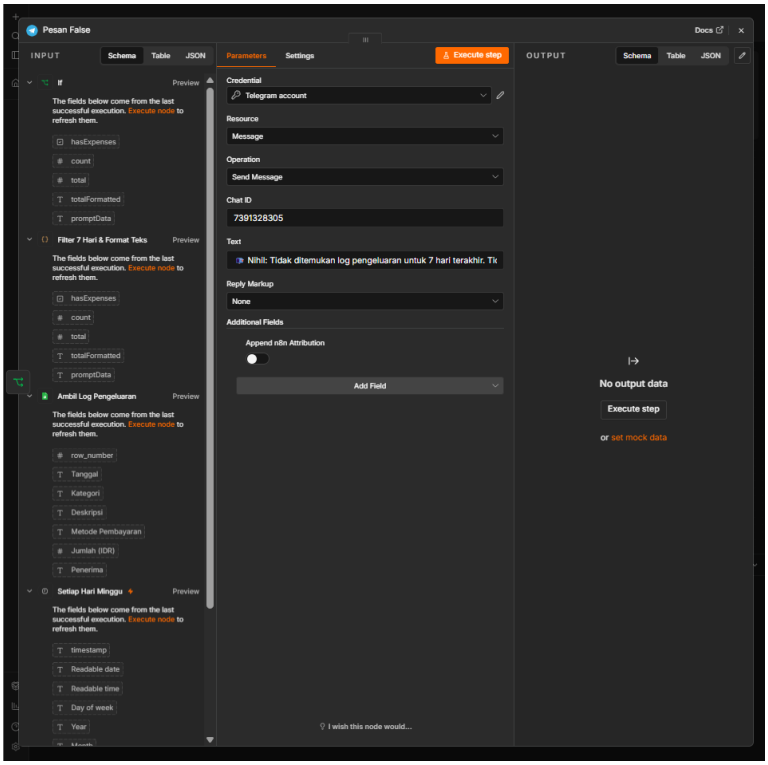
3.3 Tugas 3: Pengecekan Data dengan Node If dan Pesan "Nihil"

Node If memeriksa kondisi {{ $json.hasExpenses }} is true (Gambar 5). Bila ada transaksi dalam 7 hari terakhir, alur masuk ke cabang True menuju analisis AI. Bila tidak ada, alur masuk ke cabang False menuju node Pesan False.



Gambar 5. Konfigurasi node If (hasExpenses is true)

Node Pesan False (Telegram, operasi Send Message) mengirim pesan "Nihil: Tidak ditemukan log pengeluaran untuk 7 hari terakhir. Tidak ada analisis AI yang dibuat." (Gambar 6). Node ini tidak disambungkan ke node lain sehingga proses berhenti di sini dan tidak ada panggilan AI yang terbuang.



Gambar 6. Konfigurasi node Pesan False (Telegram)

3.4 Tugas 4: Analisis Kebiasaan Belanja dengan Google Gemini

Node Analisis (Basic LLM Chain) menerima data dari node Filter melalui {{ $json.promptData }} dan ditambah instruksi. System message menetapkan peran AI sebagai asisten keuangan pribadi (Gambar 7). Model yang digunakan adalah Google Gemini Chat Model dengan temperature 0.4 agar jawaban cukup konsisten namun tetap natural (Gambar 8).

System message:

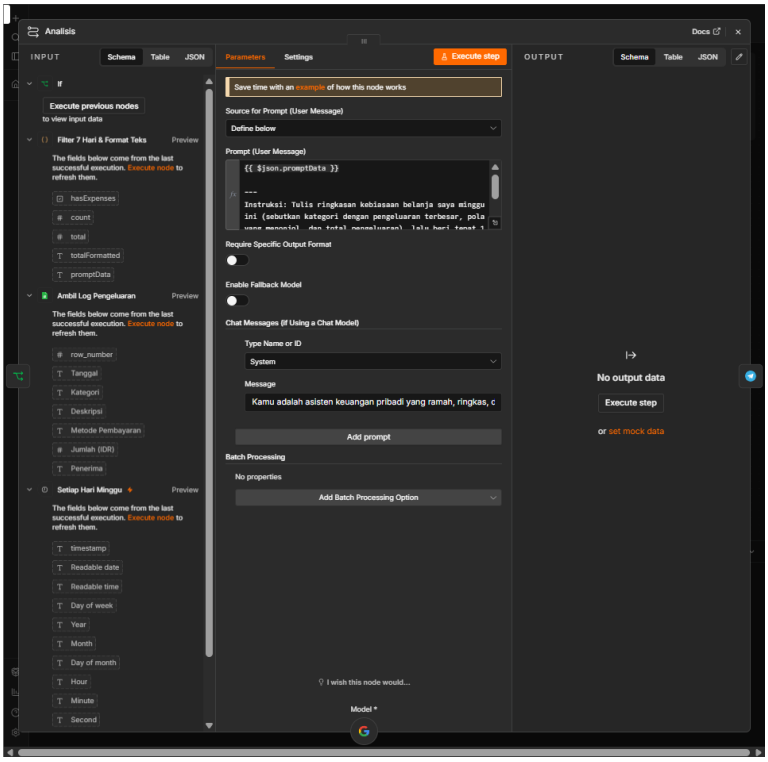
```
Kamu adalah asisten keuangan pribadi yang ramah, ringkas, dan praktis. Tugasmu
menganalisis catatan pengeluaran mingguan pengguna dan memberi saran yang
realistis. Gunakan Bahasa Indonesia. Tulis dalam teks biasa tanpa format
markdown atau HTML (jangan pakai tanda bintang, garis bawah, pagar, atau tanda
kurung siku/sudut).
```

Prompt (User Message):

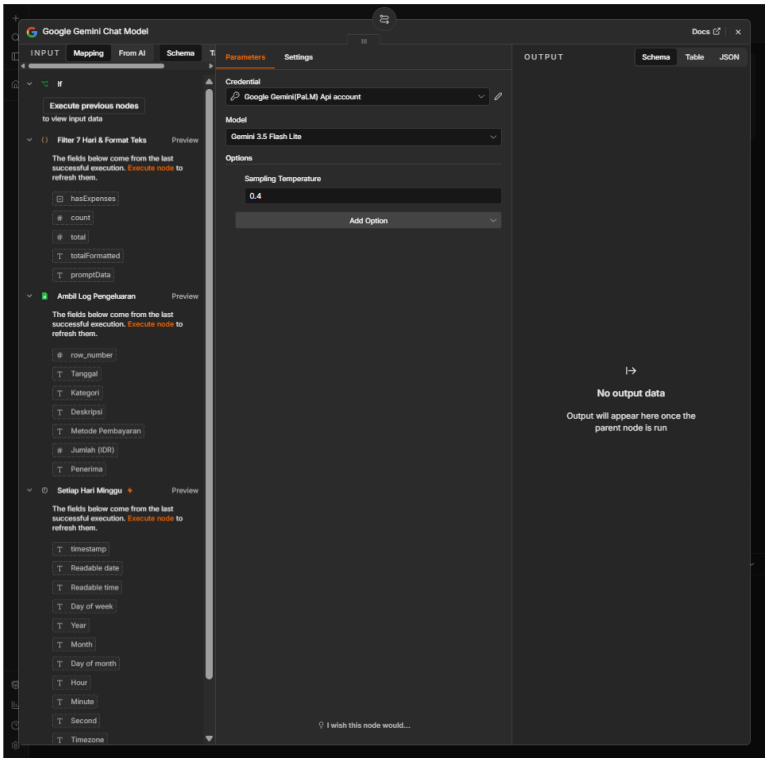
```
{{ $json.promptData }}

---
Instruksi: Tulis ringkasan kebiasaan belanja saya minggu ini (sebutkan kategori
dengan pengeluaran terbesar, pola yang menonjol, dan total pengeluaran), lalu
beri tepat 1 saran penghematan yang konkret, spesifik, dan bisa langsung
dilakukan minggu depan berdasarkan data di atas. Maksimal 150 kata.
```

Prompt terdiri dari dua bagian: data dinamis dari node sebelumnya dan instruksi statis yang menjelaskan apa yang harus dilakukan AI. Batas "tepat 1 saran" dan "maksimal 150 kata" menjaga keluaran tetap ringkas dan sesuai tugas.



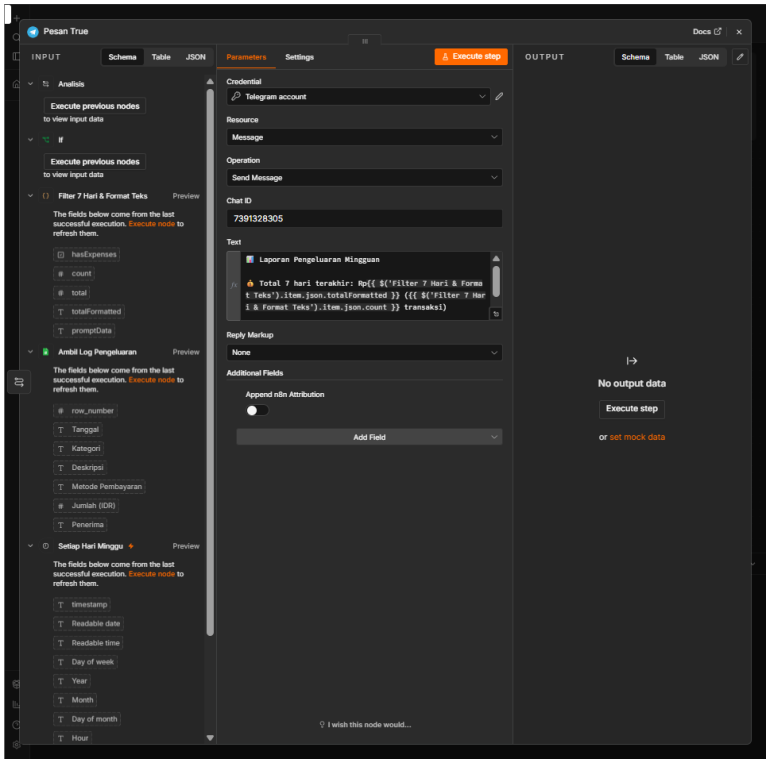
Gambar 7. Konfigurasi node Analisis (prompt dan system message)



Gambar 8. Konfigurasi Google Gemini Chat Model (Gemini Flash Lite, temperature 0.4)

3.5 Tugas 5: Pengiriman Hasil ke Telegram Pribadi

Node Pesan True (Telegram, operasi Send Message) mengirim laporan ke chat pribadi pengguna melalui Chat ID. Teks pesan terdiri dari judul laporan, total pengeluaran 7 hari terakhir beserta jumlah transaksi (diambil dari node Filter), lalu teks analisis dan saran dari AI ({{ $json.text }}) (Gambar 9). Opsi Append n8n Attribution dimatikan agar pesan bersih.



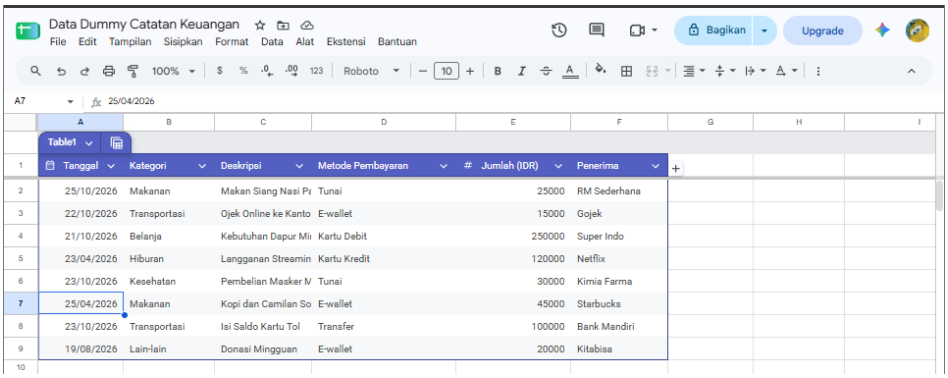
Gambar 9. Konfigurasi node Pesan True (Telegram)

4. Pengujian

Pengujian dilakukan dengan dua skenario yang mengubah tanggal pada data di Google Sheets, sehingga kedua cabang percabangan terbukti berjalan.

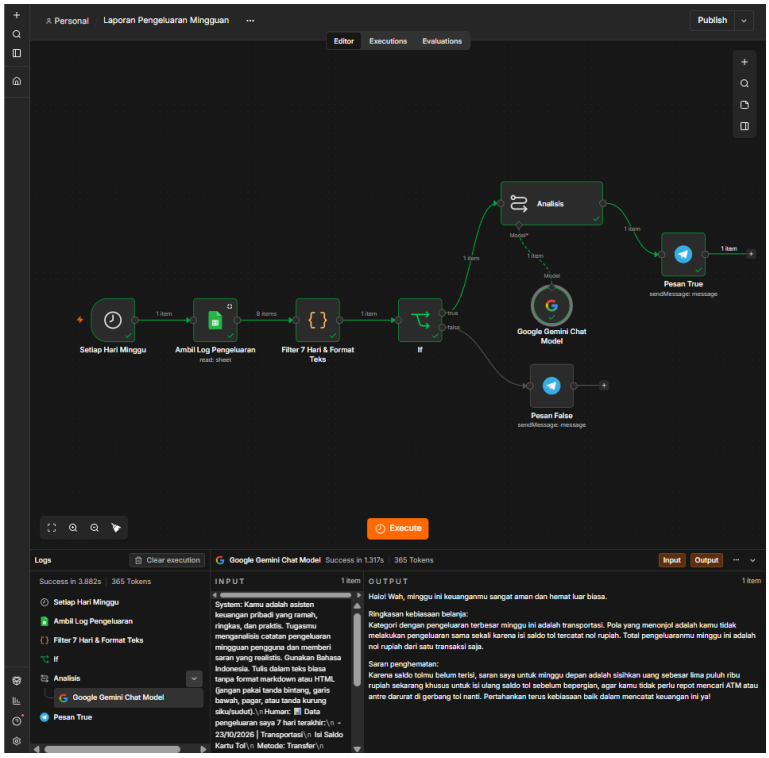
4.1 Skenario 1: Ada Pengeluaran dalam 7 Hari Terakhir (Jalur True)

Tanggal pada sheet diatur sehingga lima transaksi berada pada 21–25 Oktober 2026, sedangkan tiga lainnya (23/04/2026, 25/04/2026, dan 19/08/2026) sengaja dibiarkan lama untuk membuktikan filter bekerja (Gambar 10). Lima transaksi yang masuk rentang bernilai Rp25.000, Rp15.000, Rp250.000, Rp30.000, dan Rp100.000, dengan total Rp420.000.



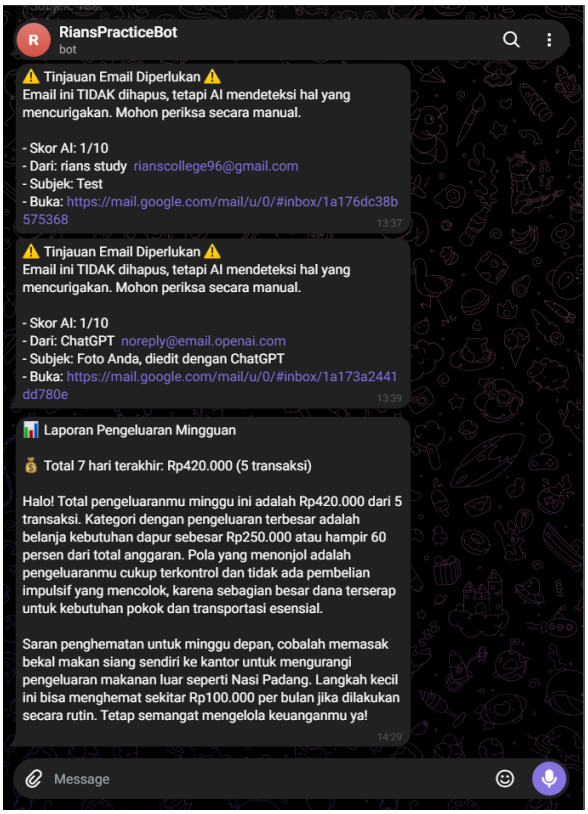
Gambar 10. Data sheet untuk Skenario 1 (lima transaksi dalam 7 hari terakhir)

Pada eksekusi awal, alur sudah masuk ke cabang True dan seluruh node (Analisis, Gemini, Pesan True) berjalan sukses (Gambar 11). Namun total masih terbaca Rp0 karena fungsi pembacaan kolom belum mengenali header "Jumlah (IDR)". Masalah ini diperbaiki (lihat bagian 4.4) dan pengujian diulang.



Gambar 11. Eksekusi awal jalur True: seluruh node berjalan, namun total masih Rp0 (sebelum perbaikan)

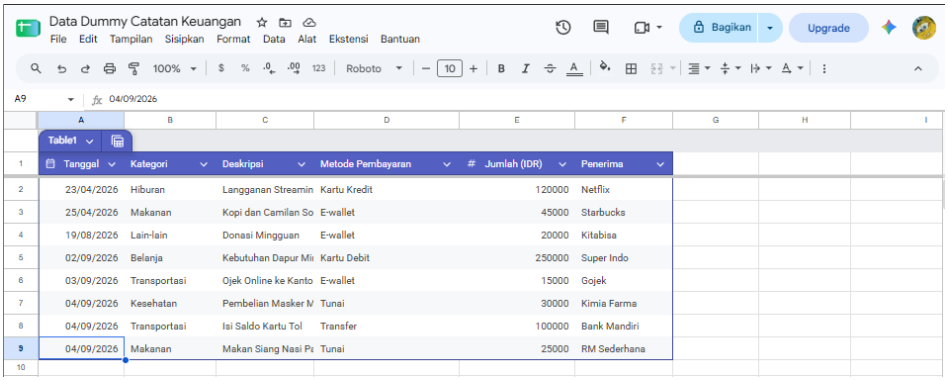
Setelah perbaikan, Telegram menerima laporan dengan total Rp420.000 dari 5 transaksi. AI menyebut belanja kebutuhan dapur sebesar Rp250.000 (sekitar 60 persen dari total) sebagai kategori terbesar, lalu memberi satu saran penghematan: membawa bekal makan siang ke kantor untuk mengurangi pengeluaran makanan luar (Gambar 12).



Gambar 12. Laporan mingguan yang diterima di Telegram setelah perbaikan

4.2 Skenario 2: Tidak Ada Pengeluaran dalam 7 Hari Terakhir (Jalur False)

Seluruh tanggal pada sheet diubah ke tanggal lama, yaitu antara 23/04/2026 dan 04/09/2026 (Gambar 13), sehingga tidak ada transaksi dalam 7 hari terakhir.



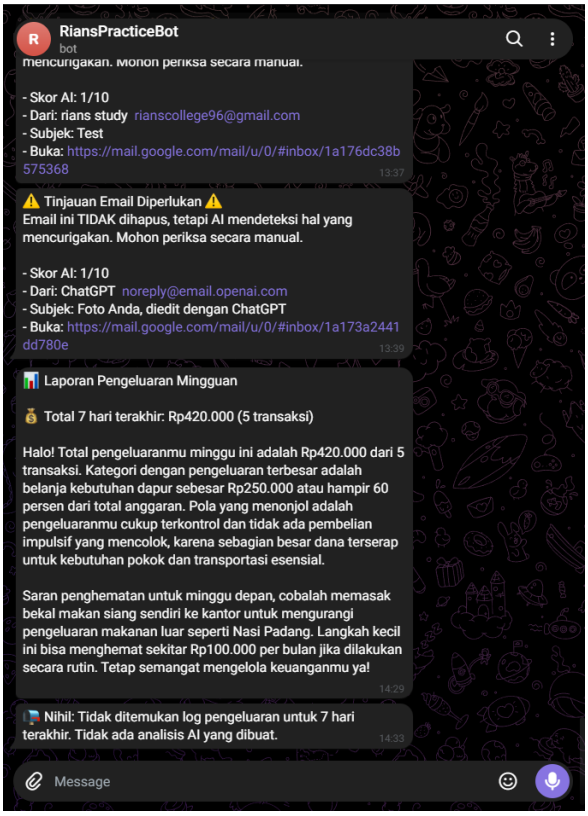
Tanggal	Kategori	Deskripsi	Metode Pembayaran	Jumlah (IDR)	Penerima
23/04/2026	Hiburan	Langganan Streamin	Kartu Kredit	120000	Netflix
25/04/2026	Makanan	Kopi dan Camilan So	E-wallet	45000	Starbucks
19/08/2026	Lain-lain	Donasi Mingguan	E-wallet	20000	Kitabisa
02/09/2026	Belanja	Kebutuhan Dapur Mi	Kartu Debit	250000	Super Indo
03/09/2026	Transportasi	Ojek Online ke Kanto	E-wallet	15000	Gojek
04/09/2026	Kesehatan	Pembelian Masker M	Tunai	30000	Kimia Farma
04/09/2026	Transportasi	Isi Saldo Kartu Tol	Transfer	100000	Bank Mandiri
04/09/2026	Makanan	Makan Siang Nasi Pa	Tunai	25000	RM Sederhana

Gambar 13. Data sheet untuk Skenario 2 (semua tanggal di luar 7 hari terakhir)

Node If mengarahkan alur ke cabang False sehingga hanya Pesan False yang dijalankan, sedangkan Analisis dan Pesan True tidak aktif (Gambar 14). Telegram menerima pesan "Nihil" (Gambar 15).



Gambar 14. Eksekusi jalur False: hanya Pesan False yang berjalan



Gambar 15. Pesan "Nihil" diterima di Telegram (di bawah laporan Skenario 1)

4.3 Ringkasan Hasil

Skenario	Kondisi Data	Transaksi Terfilter	Cabang If	Pesan di Telegram
1	Lima transaksi pada 21–25 Okt 2026	5 (Rp420.000)	True	Laporan, analisis AI, dan satu saran penghematan
2	Semua tanggal lama (Apr–Sep 2026)	0	False	Pesan "Nihil" tanpa analisis AI

4.4 Kendala dan Penyelesaian

Kendala	Penyebab	Penyelesaian
Total pengeluaran terbaca Rp0	Fungsi pick() mencocokkan nama kolom secara persis ("jumlah"), padahal header sheet adalah "Jumlah (IDR)"	Pencocokan diubah menjadi berdasarkan awalan nama (startsWith); hasil uji ulang menjadi Rp420.000
Error "missing) after argument list" pada node Code	Tanda kurung penutup hilang saat mengedit satu baris kode	Baris dilengkapi menjadi keys.find((k) => k.trim().toLowerCase().startsWith(n));
Alur selalu masuk ke Pesan False	Tanggal pada sheet masih bulan April sehingga di luar rentang 7 hari terakhir; node If bekerja benar	Tanggal pada sheet disesuaikan untuk menguji tiap jalur

5. Kesimpulan

Workflow Laporan Pengeluaran Mingguan berhasil dibangun dan diuji pada kedua jalurnya. Seluruh tugas terpenuhi: Schedule Trigger mingguan memicu pengambilan seluruh data dari Google Sheets, skrip JavaScript menyaring transaksi 7 hari terakhir dengan penanganan tanggal DD/MM/YYYY, node If mengirim pesan "Nihil" dan menghentikan proses bila tidak ada data, AI Gemini menganalisis kebiasaan belanja dengan satu saran penghematan konkret, dan hasilnya diteruskan langsung ke Telegram pribadi.

Catatan dan Saran Pengembangan

- Rentang filter mengikuti materi (7 hari lalu pukul 00.00 sampai hari ini pukul 23.59) sehingga mencakup 8 tanggal termasuk hari ini; bila ingin tepat 7 tanggal, ubah pengurang menjadi 6.
- Analisis AI hanya sebaik data di sheet; kolom Kategori yang konsisten akan membuat ringkasan kebiasaan belanja lebih akurat.
- Laporan dapat dikembangkan dengan membandingkan total minggu ini dan minggu sebelumnya.

Lampiran A. Kode Node Filter 7 Hari & Format Teks

```
// Ambil semua baris dari Google Sheets
const items = $input.all();

// Rentang waktu: 7 hari lalu (00:00:00) sampai hari ini (23:59:59)
const today = new Date();
today.setHours(23, 59, 59, 999);
const sevenDaysAgo = new Date(today);
sevenDaysAgo.setDate(today.getDate() - 7);
sevenDaysAgo.setHours(0, 0, 0, 0);

// Ambil nilai kolom dengan beberapa kemungkinan nama header (tidak peka huruf besar/kecil)
const pick = (obj, names) => {
  const keys = Object.keys(obj);
  for (const n of names) {
    const k = keys.find((k) => k.trim().toLowerCase().startsWith(n));
    if (k !== undefined && obj[k] !== '' && obj[k] != null) return obj[k];
  }
  return '';
};

// Sheet memakai format DD/MM/YYYY -> harus di-parse manual agar tidak salah baca
const parseDate = (raw) => {
  const s = String(raw).trim();
  const m = s.match(/^(\d{1,2})\/(\d{1,2})\/(\d{4})/);
  if (m) {
    const [day, month, year] = [Number(m[1]), Number(m[2]), Number(m[3])];
    return new Date(year, month - 1, day);
  }
  const d = new Date(s); // cadangan: format ISO (YYYY-MM-DD)
  return isNaN(d.getTime()) ? null : d;
};

// Jumlah bisa berupa angka atau teks seperti "Rp 25.000"
const parseAmount = (raw) => {
  if (typeof raw === 'number') return raw;
  const digits = String(raw).replace(/[^0-9]/g, '');
  return digits ? Number(digits) : 0;
};

const rupiah = (n) => String(n).replace(/\B(?=(\d{3})+(?!\d))/g, '.');

// Filter 7 hari terakhir
const rows = [];
for (const item of items) {
  const rawDate = pick(item.json, ['tanggal', 'date', 'rawdate']);
  if (!rawDate) continue;
  const expenseDate = parseDate(rawDate);
  if (!expenseDate) continue;

  if (expenseDate >= sevenDaysAgo && expenseDate <= today) {
    rows.push({
      expenseDate,
      rawDate: String(rawDate),
      kategori: pick(item.json, ['kategori', 'category']) || '-',
      deskripsi: pick(item.json, ['deskripsi', 'keterangan', 'description']) || '-',
      metode: pick(item.json, ['metode', 'metode pembayaran', 'payment method', 'method']) || '-',
      penerima: pick(item.json, ['penerima', 'merchant', 'toko']) || '-',
      jumlah: parseAmount(pick(item.json, ['jumlah', 'nominal', 'amount', 'total'])),
    });
  }
}
```

```
rows.sort((a, b) => a.expenseDate - b.expenseDate);

// Format teks untuk AI
let formattedString = '📊 Data pengeluaran saya 7 hari terakhir:\n';
let total = 0;
for (const r of rows) {
  total += r.jumlah;
  formattedString +=
    `- ${r.rawDate} | ${r.kategori}\n` +
    `  ${r.deskripsi}\n` +
    `  Metode: ${r.metode}\n` +
    `  Penerima: ${r.penerima}\n` +
    `  💸 Rp${rupiah(r.jumlah)}\n\n`;
}
formattedString += `Total: Rp${rupiah(total)} (${rows.length} transaksi)`;

const hasExpenses = rows.length > 0;

return [{
  json: {
    hasExpenses,
    count: rows.length,
    total,
    totalFormatted: rupiah(total),
    promptData: hasExpenses ? formattedString : '',
  },
}];
```